



# PRS Pricing Model — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 04-April-2026

David K Kelly

**CONFIDENTIAL — PROPRIETARY**

## **Legal Notice**

### **PROPRIETARY AND CONFIDENTIAL**

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

# 1 Test Results — PRS Pricing Model (MKM-PR-001)

Tests run on **04 April 2026 at 18:43:46**.

Table 1: Test Results Summary — PRS Pricing Model (MKM-PR-001)

|             |       |
|-------------|-------|
| Total Tests | 57    |
| Passed      | 57    |
| Failed      | 0     |
| Skipped     | 0     |
| Duration    | 7.22s |

Table 2: Detailed Test Results — PRS Pricing Model

| Test Description                                    | Acceptance Criteria                                                                           | Result | Time   |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------|--------|--------|
| 1Y from Feb 2026 = last Friday May 2027.            | <code>mat == last_friday_of_month(2027, 5)</code>                                             | Pass   | 0.000s |
| 3Y from Feb 2026 = last Friday May 2029.            | <code>mat == last_friday_of_month(2029, 5);</code><br><code>mat.month == 5 (+1 more)</code>   | Pass   | 0.000s |
| Jan 2027 uses Aug 2026 roll → Nov 2029 maturity.    | <code>mat == last_friday_of_month(2029, 11)</code>                                            | Pass   | 0.000s |
| 3Y from Sep 2026 = last Friday Nov 2029 (Aug roll). | <code>mat == last_friday_of_month(2029, 11);</code><br><code>mat.month == 11</code>           | Pass   | 0.000s |
| 5Y from Feb 2026 = last Friday May 2031.            | <code>mat == last_friday_of_month(2031, 5)</code>                                             | Pass   | 0.000s |
| 5Y from Sep 2026 = last Friday Nov 2031.            | <code>mat == last_friday_of_month(2031, 11)</code>                                            | Pass   | 0.000s |
| All maturities should fall on a Friday.             | <code>mat.weekday() == 4</code>                                                               | Pass   | 0.000s |
| 3Y maturity jumps from May to Nov at the Aug roll.  | <code>may_mat.month == 5; nov_mat.month == 11 (+1 more)</code>                                | Pass   | 0.000s |
| Aug onwards uses Aug roll.                          | <code>roll == last_friday_of_month(2026, 8);</code><br><code>roll.month == 8</code>           | Pass   | 0.000s |
| December uses the same year's Aug roll.             | <code>roll.month == 8; roll.year == 2026</code>                                               | Pass   | 0.000s |
| Feb-Jul uses Feb roll.                              | <code>roll == last_friday_of_month(2026, 2);</code><br><code>roll.month == 2</code>           | Pass   | 0.000s |
| January uses the previous year's Aug roll.          | <code>roll == last_friday_of_month(2026, 8);</code><br><code>roll.month == 8 (+1 more)</code> | Pass   | 0.000s |
| July is the last month of the Feb roll period.      | <code>roll.month == 2; roll.year == 2026</code>                                               | Pass   | 0.000s |
| March is still in the Feb roll period.              | <code>roll.month == 2; roll.year == 2026</code>                                               | Pass   | 0.000s |

| Test Description                                                       | Acceptance Criteria                                                                                  | Result | Time   |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|--------|--------|
| compute_maturity_date with no ref_date uses today.                     | <code>isinstance(mat, date); mat.weekday() == 4</code>                                               | Pass   | 0.000s |
| current_roll_date(None) uses today and returns a valid date.           | <code>isinstance(roll, date); roll.weekday() == 4</code>                                             | Pass   | 0.000s |
| maturity_table with no ref_date uses today.                            | <code>len(table) == 5; all(isinstance(e['maturity_date'], date) for e in table)</code>               | Pass   | 0.000s |
| Empty propertyts dir                                                   | <code>stats["total_properties"] == 0; stats["properties_processed"] == 0</code>                      | Pass   | 0.013s |
| Generator should still work without gauge hazard curves (empty basis). | <code>stats["properties_processed"] == 3; stats["properties_skipped"] == 0 (+1 more)</code>          | Pass   | 0.281s |
| No propertyts dir                                                      | —                                                                                                    | Pass   | 0.000s |
| Spread is flat term structure                                          | <code>all(s == spreads[0] for s in spreads)</code>                                                   | Pass   | 0.266s |
| Spread proportional to floods                                          | <code>spread3 &gt; spread1 &gt; spread2</code>                                                       | Pass   | 0.319s |
| Zero floods zero spread                                                | <code>spreads[0] == expected</code>                                                                  | Pass   | 0.264s |
| Returns severe event count                                             | <code>gen._get_gauge_severe_count(gauge_hc) == 7</code>                                              | Pass   | 0.000s |
| Returns zero for empty dict                                            | <code>gen._get_gauge_severe_count({}) == 0</code>                                                    | Pass   | 0.000s |
| Returns zero when missing                                              | <code>gen._get_gauge_severe_count(gauge_hc) == 0</code>                                              | Pass   | 0.000s |
| Aug 2026: 31st is Monday, last Friday = 28th.                          | <code>last_friday_of_month(2026, 8) == date(2026, 8, 28)</code>                                      | Pass   | 0.000s |
| Feb 2026: 28th is Saturday, last Friday = 27th.                        | <code>last_friday_of_month(2026, 2) == date(2026, 2, 27)</code>                                      | Pass   | 0.000s |
| May 2029: 31st is Thursday, last Friday = 25th.                        | <code>last_friday_of_month(2029, 5) == date(2029, 5, 25)</code>                                      | Pass   | 0.000s |
| Nov 2029: 30th is Friday.                                              | <code>last_friday_of_month(2029, 11) == date(2029, 11, 30)</code>                                    | Pass   | 0.000s |
| Result should always be a Friday (weekday=4).                          | <code>d.weekday() == 4</code>                                                                        | Pass   | 0.000s |
| Actual years should be strictly increasing.                            | <code>years[i] &gt; years[i - 1]</code>                                                              | Pass   | 0.000s |
| All maturities should be in May for the Feb roll.                      | <code>e['maturity_date'].month == 5</code>                                                           | Pass   | 0.000s |
| All maturities should be in November for the Aug roll.                 | <code>e['maturity_date'].month == 11</code>                                                          | Pass   | 0.000s |
| Maturity string should be formatted as 'DD Mon YYYY'.                  | <code>len(e['maturity_str']) &gt; 8; 'May' in e['maturity_str'] or 'Nov' in e['maturity_str']</code> | Pass   | 0.000s |

| Test Description                                                              | Acceptance Criteria                                                                                                                    | Result | Time   |
|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------|--------|
| Default table has 5 entries (1Y through 5Y).                                  | <code>len(table) == 5</code>                                                                                                           | Pass   | 0.000s |
| Tenors should be 1, 2, 3, 4, 5.                                               | <code>tenors == [1, 2, 3, 4, 5]</code>                                                                                                 | Pass   | 0.000s |
| Basis calculation                                                             | <code>len(nearest) &gt; 0; ng0["gauge_id"] == "GAUGE-001" (+5 more)</code>                                                             | Pass   | 0.288s |
| Depth thresholds                                                              | <code>"severe" in thresholds; "annual_probability" in thresholds["severe"] (+1 more)</code>                                            | Pass   | 0.264s |
| Generate produces output                                                      | <code>output_path.exists(); "metadata" in data (+2 more)</code>                                                                        | Pass   | 0.276s |
| Gev params always none                                                        | <code>pc["gev_params"] is None</code>                                                                                                  | Pass   | 0.272s |
| PROP-003 has 10 floods, nearest gauge has 8 severe events.                    | <code>prop3["summary"]["flood_transmission_rate"] &gt; 0</code>                                                                        | Pass   | 0.261s |
| Output structure                                                              | <code>"PROP-001" in curves; "PROP-002" in curves (+13 more)</code>                                                                     | Pass   | 0.286s |
| Properties processed                                                          | <code>stats["properties_processed"] == 3; stats["properties_skipped"] == 0</code>                                                      | Pass   | 0.259s |
| With event count pricing, term structure spread is flat (storms independent). | <code>s == spreads[0]</code>                                                                                                           | Pass   | 0.258s |
| Term structure                                                                | <code>ts["tenors"] == TENORS; "severe" in ts (+2 more)</code>                                                                          | Pass   | 0.356s |
| After attach_spread_decomposition, propertyhc.json has decomposition.         | <code>"spread_decomposition" in pc; "gauge_spread_bps" in sd (+5 more)</code>                                                          | Pass   | 0.780s |
| At least one effect per path must be non-zero when gauge != property.         | <code>df["distance_effect_bps"] != 0.0 or df["elevation_effect...; ef["elevation_effect_bps"] != 0.0 or ef["distance_effect...]</code> | Pass   | 0.840s |
| Both decomposition paths should arrive at the property spread.                | <code>abs(path1 - prop) &lt; 0.1; abs(path2 - prop) &lt; 0.1</code>                                                                    | Pass   | 0.803s |
| With shd/she files present, decomposition values must be non-zero.            | <code>sd["property_spread_bps"] != 0.0; sd["shd_spread_bps"] != 0.0 (+1 more)</code>                                                   | Pass   | 0.864s |
| Without shd/she files, decomposition still works (zeros).                     | <code>n &gt; 0; sd["shd_spread_bps"] == 0.0 (+1 more)</code>                                                                           | Pass   | 0.264s |
| <code>_get_5yr_spread</code> must return index 4 when available.              | <code>PropertyHazardCurveGenerator._get_5yr_spread() == 50.0</code>                                                                    | Pass   | 0.000s |
| <code>_get_5yr_spread</code> must return 0.0 for empty/missing data.          | <code>PropertyHazardCurveGenerator._get_5yr_spread() == 0.0; PropertyHazardCurveGenerator._get_5yr_spread(... (+1 more)</code>         | Pass   | 0.000s |

| Test Description                                                 | Acceptance Criteria                                                                                                                                     | Result | Time   |
|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|--------|--------|
| <code>_get_5yr_spread</code> must work with specified threshold. | <code>PropertyHazardCurveGenerator._get_5yr_spread(pc, 'severe'...;</code><br><code>PropertyHazardCurveGenerator._get_5yr_spread(pc, 'any_flo...</code> | Pass   | 0.000s |
| All tenors should round-trip through maturity and back.          | <code>tenor_from_maturity(mat, ref) == t</code>                                                                                                         | Pass   | 0.000s |
| A date not in the table returns the closest tenor.               | <code>result in [1, 2, 3, 4, 5]</code>                                                                                                                  | Pass   | 0.000s |
| Exact maturity date returns correct tenor.                       | <code>tenor_from_maturity(mat, date(2026, 2, 24)) == 3</code>                                                                                           | Pass   | 0.000s |

## 1.1 Test Document History

| Date        | Version | Description                                      | Author   |
|-------------|---------|--------------------------------------------------|----------|
| 05-Mar-2026 | 1.0     | Initial test suite (30 tests)                    | D. Kelly |
| 07-Mar-2026 | 1.1     | 57 test(s) added; 30 test(s) removed (total: 57) | D. Kelly |
| 07-Mar-2026 | 1.2     | 30 test(s) added (total: 87)                     | D. Kelly |
| 18-Mar-2026 | 1.3     | 4 test(s) added (total: 91)                      | D. Kelly |
| 27-Mar-2026 | 1.4     | 30 test(s) removed (total: 61)                   | D. Kelly |
| 30-Mar-2026 | 1.5     | 3 test(s) added; 12 test(s) removed (total: 52)  | D. Kelly |
| 01-Apr-2026 | 1.6     | 4 test(s) added (total: 56)                      | D. Kelly |
| 03-Apr-2026 | 1.7     | 3 test(s) added; 2 test(s) removed (total: 57)   | D. Kelly |
| 04-Apr-2026 | 1.8     | 9 test(s) added; 9 test(s) removed (total: 57)   | D. Kelly |